

TECH SHARING • 60 MIN

多智能体系统

Multi-Agent Systems

从经典基础架构，到 2026 年的前沿协议与工程实践

面向技术同事 · 涵盖 MCP / A2A / ACP / ANP 与主流编排框架





AGENDA · 本次分享路线

六个部分：从动机到协议，再到 2026 前沿



01

为什么需要多智能体

单 Agent 的边界 · 分而治之的收益与代价



02

经典基础与里程碑论文

CAMEL · MetaGPT · 生成式智能体 · Debate · MoA



03

架构、编排与组织

拓扑 · 编排开销 · 智能体组织



04

主流框架全景

LangGraph · CrewAI · AutoGen · ADK · SDK



05

智能体协议与 Agent 经济

MCP · A2A · ANP + 支付层 AP2 · x402 (重点)



06

2026 前沿、经验与评测

MAST 失败 · 工业界经验 · 评测与避坑

01



WHY MULTI-AGENT

为什么需要多智能体？

From a single reasoning loop to a society of agents





01 · 智能体的本质

什么是 Agent: 以 LLM 为大脑的「感知-推理-行动」循环

ReAct 循环 (Yao et al., 2022)



推理内核 Reasoning

LLM 负责规划、决策与反思



记忆 Memory

短期上下文 + 长期向量/外部存储



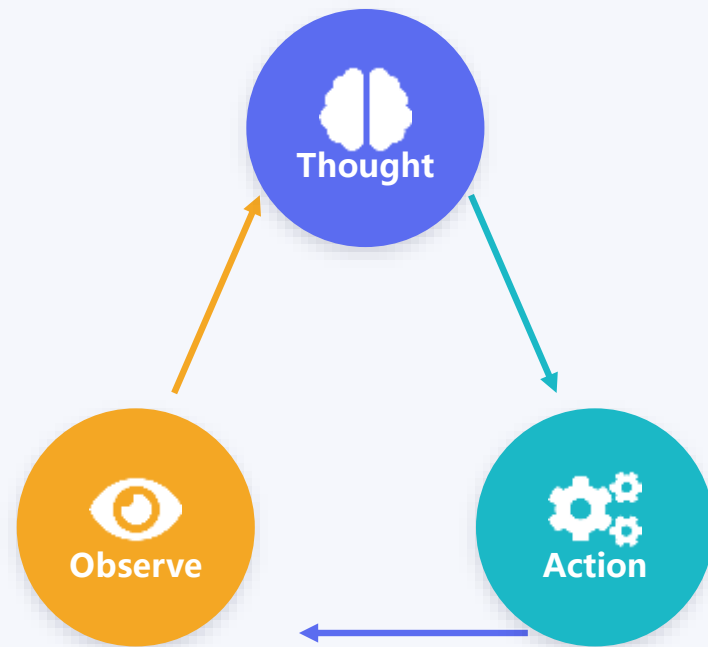
工具 Tools

调用 API、检索、代码执行、浏览器



感知与行动 Act

观察环境反馈，产生下一步动作



思考 → 调用工具 → 读取结果 → 再思考.....直至完成



具体示例：「帮我订下周去上海最便宜的高铁票」——Agent 先**思考**(拆解：查日期→查车次→比价)，再**行动**(调用12306查询工具)，**观察**返回结果并选出最优，最后下单。每一步都由 LLM 实时决策。



为什么要多个 Agent? 分而治之的收益——与必须正视的代价



单 Agent 的瓶颈

Single-agent limits

- ✗ **上下文窗口有限** 长程任务塞不下全部信息，越走越「健忘」
- ✗ **单一视角** 缺少交叉检验，易确认偏误、一条道走到黑
- ✗ **串行、慢** 只能一步步来，无法并行探索多条路径
- ✗ **工具/职责过载** 一个提示里堆几十个工具，选择与可靠性下降



多 Agent 的收益

Multi-agent gains

- ✓ **分而治之** 把大任务拆给专精子 Agent，各自独立上下文
- ✓ **并行加速** 多个子 Agent 同时探索，广度优先更快
- ✓ **专业分工** 研究员/编码/审查/批判等角色各司其职
- ✓ **可组合 & 解耦** 不同框架/厂商的 Agent 可拼装、可替换



诚实的前提：多 Agent 不是「越多越好」。真正逼我们拆分的根因是「上下文窗口有限、任务无限」。能把整个任务装进一个上下文的单 Agent，往往更稳更省——拆分是要付协调成本的。第 6 节会用 MAST 数据正面回应。

02



FOUNDATIONS & MILESTONES

经典基础与里程碑论文

From BDI/FIPA to the LLM-agent papers that defined the field





02 · 思想并非全新

前 LLM 时代：多智能体的核心思想，几十年前就已奠基



BDI 模型

Belief-Desire-Intention (1990s)

用「信念-愿望-意图」刻画理性 Agent 的内部状态与决策；至今仍是 Agent 心智建模的经典框架。



ACL: KQML / FIPA-ACL

Agent 通信语言 (1990s-2000s)

定义 Agent 之间消息的「言语行为」（inform / request / propose...）。这正是今天 A2A、ACP 协议的精神先驱。



合同网协议

Contract Net Protocol (1980)

「招标-投标-中标」式任务分配：管理者广播任务，工作者竞标，择优委派——与今天的 orchestrator 调度同构。



一句话承上启下：通信、协商、分工、招投标——这些机制早已成熟。LLM 真正改变的是「Agent 内核」：**用自然语言推理替代了硬编码规则**，让通用、可对话、能即时规划的 Agent 第一次成为现实。



02 · 里程碑时间线

LLM 智能体的奠基性工作 (2022 → 2024)



两条演化主线：① 让单个 Agent「会推理、能反思」(ReAct→Reflexion)；② 让多个 Agent「能协作、可分工」(CAMEL→AutoGen/MetaGPT)。两条线最终汇合，催生了今天的多智能体编排与协议。



四篇里程碑：核心思想 · 一个示例 · 为何重要



ReAct

Reason + Act (2022)

把「思考链」与「工具调用」交错：推理决定下一步动作，动作结果反哺推理。

示例 例：问答时先想「该查什么」→搜索→读结果→再想，而非一次性蒙答案。

影响 几乎所有现代 Agent 框架的底层循环，都是 ReAct 的变体。



Reflexion

Verbal RL (2023)

失败后让 Agent 用自然语言写「复盘反思」，存入记忆，指导下次重试——无需更新参数。

示例 例：代码题第一次跑错，反思「边界条件没处理」，第二次据此修正。

影响 把「自我批判 / 评审者」机制带入 Agent，是多 Agent 辩论与验证的雏形。



Generative Agents

Stanford 小镇 (2023)

25 个 Agent 生活在沙盒小镇，具备记忆流、反思、规划，涌现出邀约聚会等社会行为。

示例 例：一个 Agent 决定办派对，消息在镇上自发扩散、互相协调。

影响 证明了 Agent 群体可产生「涌现」的可信社会行为，激发大量仿真研究。



MetaGPT

SOP 软件公司 (2023)

把人类「标准作业流程 SOP」注入多 Agent：产品/架构/工程/测试各司其职、按流程交付。

示例 例：一句需求 → 自动产出 PRD、设计、代码、测试，像一家迷你软件公司。

影响 代表「角色化 + 流程化」范式，与 ChatDev、AutoGen 共同定义了协作框架。



按主题速览：协作范式 · 组织 · 编排 · 协议 · 评测



协作范式

- **Multi-Agent Debate** 辩论 · Du 2023
- **Mixture-of-Agents** 分层聚合 · 2024
- **CAMEL** 角色扮演双 Agent · 2023



角色与分工

- **MetaGPT** SOP 软件公司 · 2023
- **ChatDev** 对话式软件团队
- **AutoGen** GroupChat 群聊编排



社会与组织

- **Generative Agents** 斯坦福小镇 · Park 2023
- **Agent Society** Agent 社会 / 组织建模
- **TheAgentCompany** 数字员工评测 · 2024



编排与拓扑

- **Orchestrator-Worker** 主管-工作者（主流）
- **Hierarchical** 层级 / 监督者拓扑
- **Swarm / Network** 去中心化群体



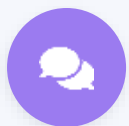
通信与协议

- **KQML / FIPA-ACL** 早期智能体通信语言
- **MCP** Agent ↔ 工具 · 2024
- **A2A** Agent ↔ Agent · 2025



失败与评测

- **MAST** 失败模式实证 · 2025
- **GAIA / AgentBench** 通用 / 多域评测
- **τ-bench** 可靠性 pass^k · 2024



02 · 深读（扩展二）

多体协作有用吗？证据、收益与边界



Multi-Agent Debate · 多智能体辩论

Du et al. 2023 · 思想源自 Minsky 《Society of Mind》

多个 LLM 实例对同一问题各自作答，再读取彼此答案、多轮互相批判与修正 → 提升事实准确性与数学推理。

GSM8K / 事实性 ↑



Mixture-of-Agents · 分层混合

Wang et al. · Together AI · 2024

分层结构：多个 proposer 各自生成 → aggregator 融合精炼，逐层提升；纯提示、无需微调。

仅用开源模型 **65.1%** > GPT-4o 57.5% (AlpacaEval 2.0)



但要清醒：协作不是免费的

辩论 ≈ 投票一致性

Huang et al. 2023 《LLMs Cannot Self-Correct Reasoning Yet》：同等算力下，多体辩论仅略胜、常不及 self-consistency 多数投票。

收益常来自「多采样 + 聚合」

很多「协作收益」其实源于多次采样再聚合，与是否「多智能体」无必然关系。

失败多是工程问题

MAST 实证：多体常因角色 / 沟通 / 验证等协作工程而失败，而非模型能力不足。

成本不可忽视

多体的 token 开销可达单体的数倍至十几倍，延迟也更高。



结论 协作在 事实核验 / 多样化生成 / 读密集 任务上有真实收益；但务必对照「等算力的简单基线」（多数投票 / 单体强提示），否则容易把「更贵」误当「更强」。

03



ARCHITECTURE • ORCHESTRATION • ORGANIZATION

架构、编排与智能体组织

Components, orchestration patterns, and agent organizations





03 · 解剖一个 MAS

多智能体系统的五大核心构件



角色 / Profile

每个 Agent 的身份、目标与权限边界（如研究员、编码者、审查者）

persona / role



规划 Planning

任务分解、子目标排序、动态重规划（CoT / ToT / 反思）

decompose & plan



记忆 Memory

短期上下文 + 长期向量库 + 共享黑板/scratchpad

short & long-term



工具 Tools

检索、API、代码执行、浏览器——多通过 MCP 暴露

via MCP



通信 Communication

消息传递、任务委派、状态同步——多通过 A2A

via A2A



为什么要协议？ 注意最右两块——「工具」与「通信」恰好对应今天两大协议：Agent↔工具 = **MCP**，Agent↔Agent = **A2A**。第 5 节将深入剖析。这是从「架构」过渡到「协议」的关键接口。



四种主流协作拓扑：谁来决定「下一个谁说话」

串行 / 流水线

Sequential



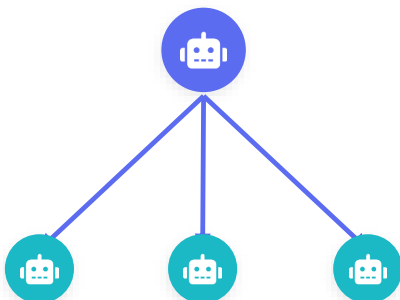
A→B→C 依次传递，像装配线

✓ 简单可控

✗ 无并行

主管-工作者

Orchestrator-Worker



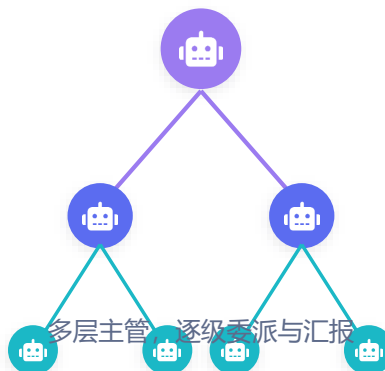
主管调度 + 子 Agent 并行干活

✓ 并行/清晰

✗ 主管瓶颈

层级 / 树

Hierarchical



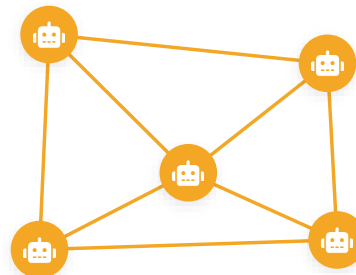
多层主管，逐级委派与汇报

✓ 可扩展

✗ 链路长

网状 / 群体

Network · Swarm



Agent 直接互相通信、共享状态

✓ 灵活

✗ 难推理/调试

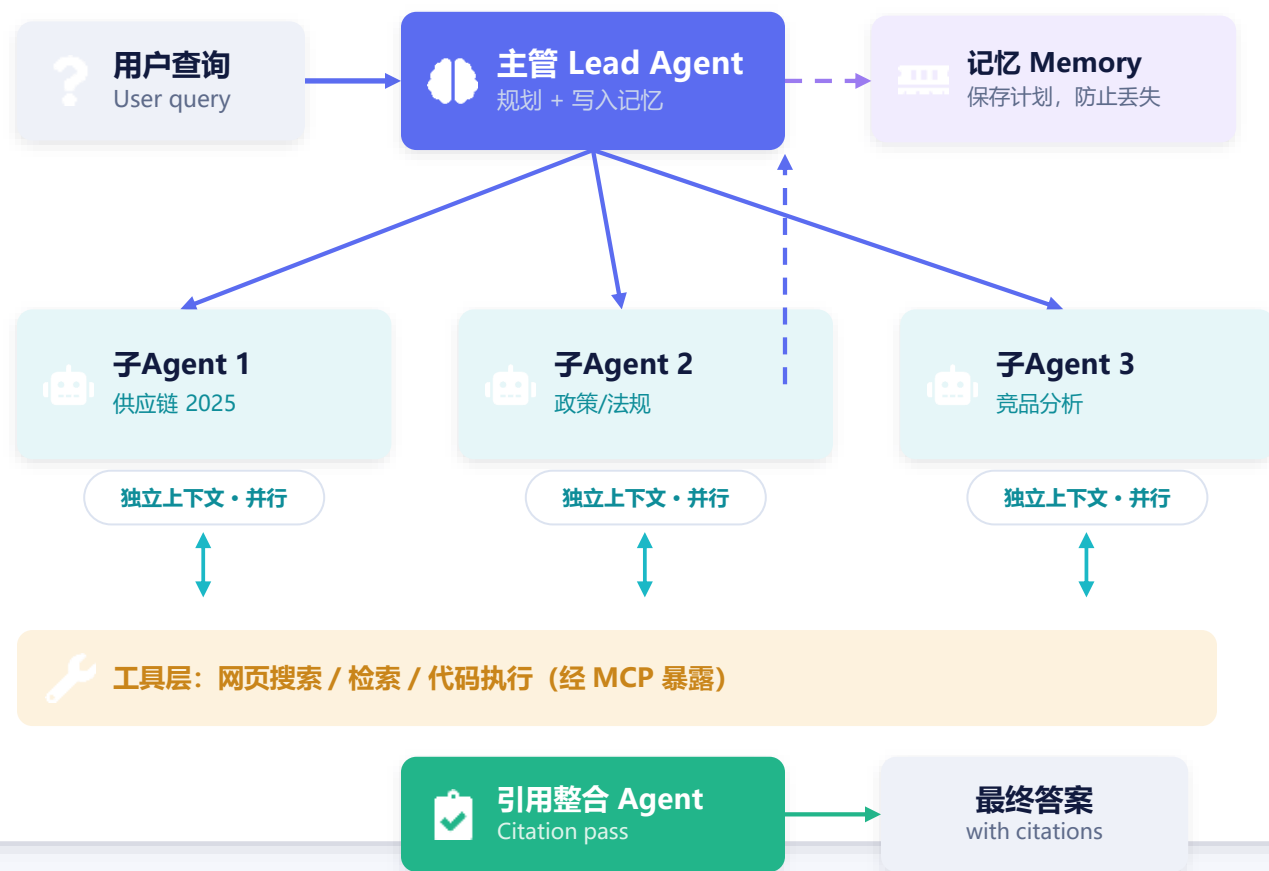


选型直觉：拓扑越「受约束」越好调试。生产里最稳的是**主管-工作者**（工人之间不互相通信，所有路由决策都在主管处）；网状最灵活但最难保证正确性。还有第五种：**辩论 / 投票 (Debate)**——多个 Agent 各自作答再相互批判取共识，常用于提升推理可靠性。



Orchestrator-Worker: Anthropic 多智能体研究系统

架构：主管规划 → 并行子 Agent → 引用整合



关键数据

+90.2%

内部研究评测上, 较单体 Claude Opus 4 的提升

≈ 15× 的 token 成本——能力的代价

四条工程经验

- 1 每个子 Agent 独立上下文窗口 各跑各的线索, 互不污染
- 2 明确委派 给目标、边界、输出格式、用哪些工具
- 3 努力分级 Effort scaling 简单核查 1 个 Agent; 复杂对比 2-4 个
- 4 工人之间不互相通信 所有路由决策都收敛在主管处



多智能体编排的工程现实：开销、延迟与可落地的杠杆

两种协调风格

编排 Orchestration (中心) 主管掌控流程、集中决策与聚合 → 可控、易审计，但主管是瓶颈

编舞 Choreography (去中心) 事件驱动、各 Agent 自主响应 → 松耦合、可扩展，但更难调试

协调不是免费的

Token 额外开销 独立式 $\approx +58\%$ · 中心式 $\approx +285\%$

协调延迟 三层级 $\times 2s/\text{层} \approx 6s$ 才轮到 worker 开工

通信收益递减：约 0.39 消息 / 轮之后趋于平缓



2026 生产现实

● Orchestrator-Worker $\approx 70\%$

主流生产部署都是「主管-工作者」

● 约 2/3 的 agentic 市场

已从单体转向多体协作

● 实用团队规模 3-4 个 Agent

再多，协调开销陡增、收益递减



降本提效的杠杆



能力感知路由

按专长 / 模型分级派活（强模型只接难题）



异步编排

不阻塞等待同步步骤，降低端到端延迟



稀疏通信

减少冗余消息，避免全员广播



层级分解

嵌套小团队，限定预算与作用域



把 Agent 编成「团队 / 公司」：结构、规范与现实

三种组织结构



平级 / 对等 Equi-level

同级协作 / 谈判 / 辩论



层级 Hierarchical

领导规划、下属执行



嵌套 Nested

团队中再含团队，分形分解

对应博弈：对等=协商 / 博弈，层级=Stackelberg 领导-跟随

组织要素与谱系

角色 Roles

规范 Norms

流程 SOP

共享记忆

代表系统

- **MetaGPT** 角色化 + SOP 的「软件公司」
- **ChatDev** 对话式开发流水线
- **Generative Agents** 小镇社会的涌现行为

学术谱系 OperA / JaCaMo——以角色、规范、组织结构形式化协作。

核心挑战 长程组织一致性：跨层级 / 时间 / 依赖维持连贯状态。



现实检验

TheAgentCompany

CMU · NeurIPS 2024/25

模拟软件公司 + 16 个 AI 同事；真实工具 GitLab / Plane / ownCloud / RocketChat；175 任务跨 SDE / PM / HR / 财务。

~30%

最强模型自主完成率（社区可达 ~43%）

社交 + 多工具任务最难：SDE ~42% > HR ~18% / 财务 ~22%。

结论：简单活能干，长程 + 社交协作仍远未自动化。

04



FRAMEWORK LANDSCAPE

主流框架全景

LangGraph · CrewAI · AutoGen · OpenAI / Google / Anthropic SDKs





六大主流框架：编排模型 · 状态 · 模型绑定 · 最适场景

框架	编排模型	状态 / 持久化	模型绑定	最适场景
LangGraph	有向图 + 条件边	内置 checkpoint / 时间旅行	无关	生产级、强控制、可审计的有状态 workflows
CrewAI	角色制 Crew + 流程	任务输出顺序传递	无关	快速原型，需求能清晰映射到「角色+任务」
AutoGen / AG2	对话式 GroupChat	对话历史（默认内存）	无关	研究、多 Agent 辩论与验证、人工审阅
OpenAI Agents SDK	显式 Handoff 交接	上下文变量（默认易失）	OpenAI	GPT 原生、低摩擦、需沙箱工具与子 Agent
Google ADK	层级 Agent 树	会话状态（可插拔后端）	偏 Gemini	多模态、GCP 原生、层级编排
Claude Agent SDK	工具链 + 子 Agent	经 MCP / 文件系统	Claude	代码优先（驱动 Claude Code）、Agentic 检索



共识： 六大框架已全部支持 MCP 进行工具互操作；底层几乎都是 ReAct 循环的变体。框架是「库」，与之并存的还有托管「平台」（Bedrock AgentCore / Vertex Agent Builder / Copilot Studio...）——多数大型项目「平台保广度、框架保深度」并用。



04 · 选型与一个反直觉事实

怎么选？以及——「编排」可能比「选哪个模型」更重要

四条选型直觉

生产 / 强控制 / 合规

→ LangGraph 或自研编排 (约 28% 生产部署选择自研)

快速原型 / 角色清晰

→ CrewAI, 几十行起步, 最快出 Demo

研究 / 辩论 / 人工审阅

→ AutoGen-AG2, 多轮对话与验证成熟

窄场景的任务交接

→ OpenAI Agents SDK 的显式 Handoff



反直觉数据 · Princeton HAL

GAIA 基准 · 同一模型 · 仅换编排脚手架

≈ 30 分

「裸模型」与「精心设计的脚手架」之间，GAIA 绝对分差可达约 30 分——比多数前沿模型「代际」之间的差距还大。

同款 Claude Opus 4: 换一套 Agent 脚手架，GAIA 成绩相差约 7 个百分点。



结论：在很多任务上，「怎么编排」对最终效果的影响，可能大于「选哪个底座模型」。别把全部精力放在比模型分数上——编排设计、上下文管理、工具与验证同样是一等公民。

05



AGENT PROTOCOLS & ECONOMY

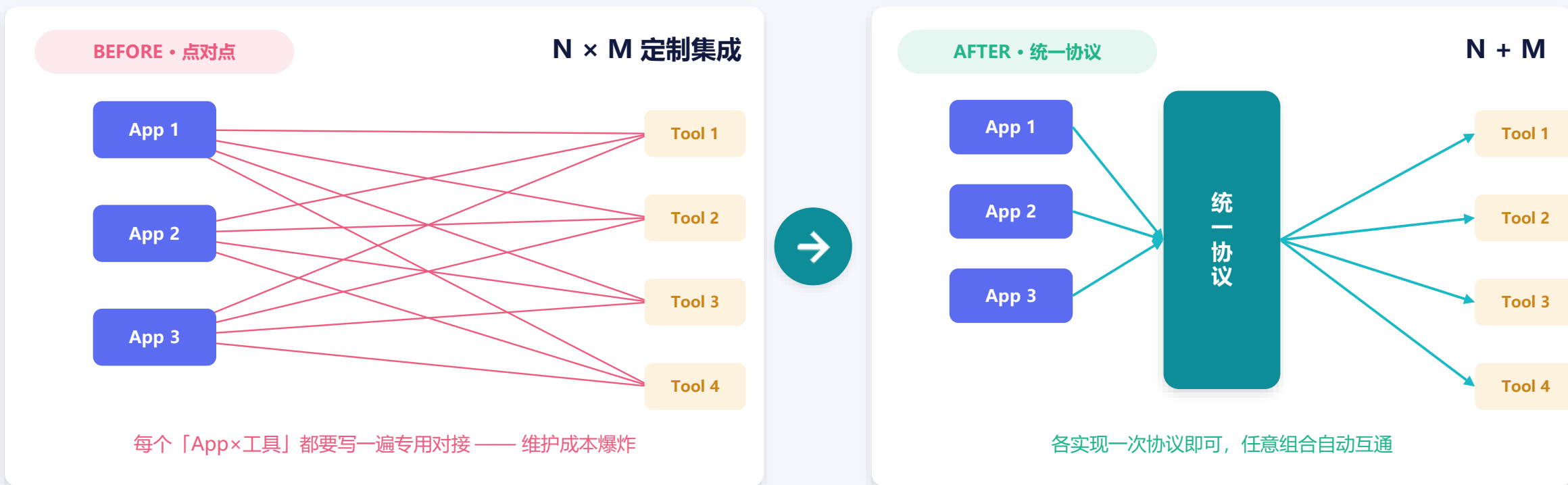
智能体协议与 Agent 经济

MCP · A2A · ACP · ANP, 及其之上的支付 / 商务层 (AP2 · x402)



05 · 为什么需要协议

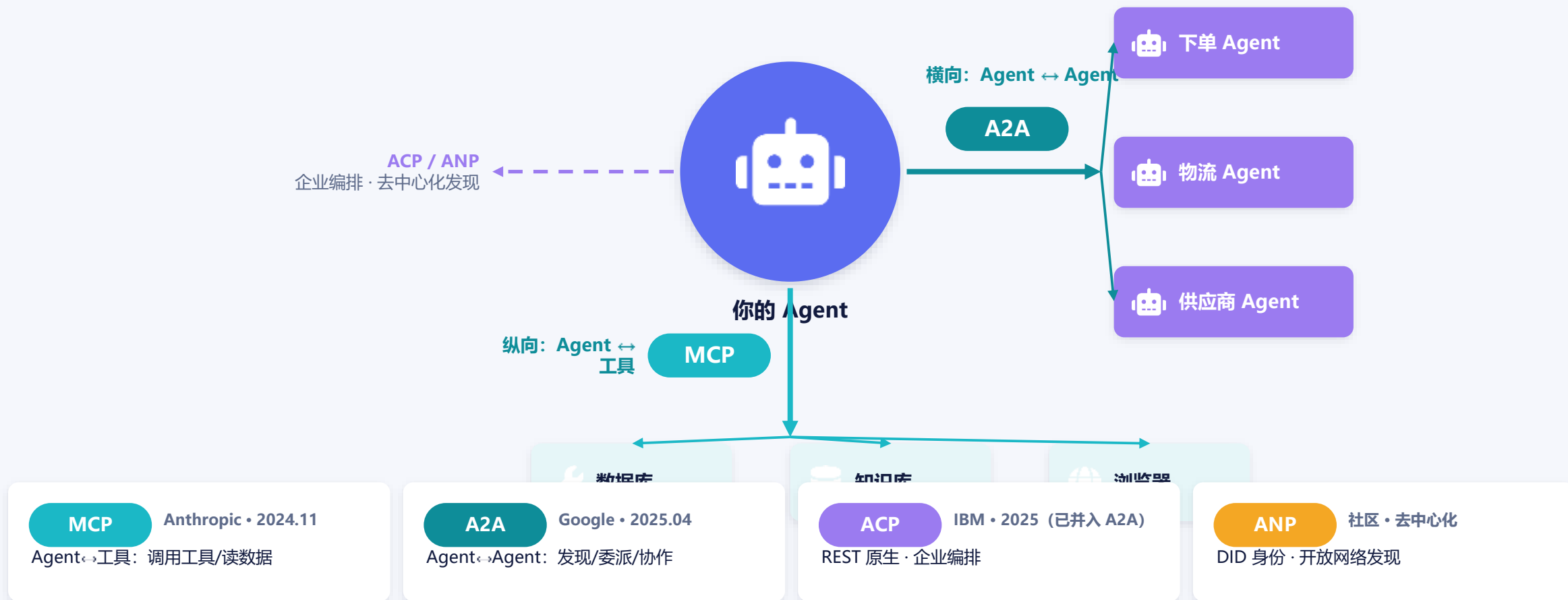
N×M 集成噩梦 → 统一协议：Agent 世界的「HTTP / USB-C」



类比：正如早期互联网需要 HTTP 把异构系统连起来，今天的「Agent 互联网」也需要统一的通信层。MCP 常被称作「AI 世界的 USB-C」—— 一个接口，连接一切工具。



两个维度看懂协议全景：纵向连工具，横向连 Agent





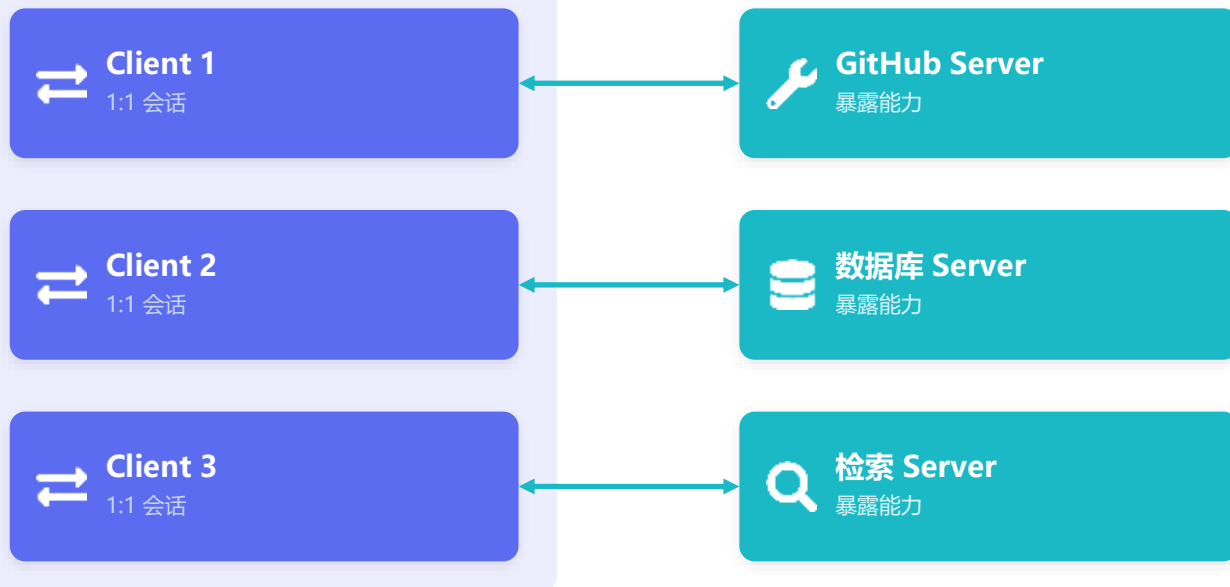
05 · MCP 深讲 (一)

架构与三大原语: Host-Client-Server + Tools/Resources/Prompts

Host - Client - Server (基于 JSON-RPC 2.0)

Host 宿主应用

Claude Desktop · IDE · Cursor



每个 Client 与 一个 Server 保持独立有状态会话

Server 暴露的三原语



Tools 工具

可执行动作 (模型决定调用)



Resources 资源

只读上下文数据 (如文件、记录)



Prompts 提示

可复用的提示模板 / 工作流

两种传输方式

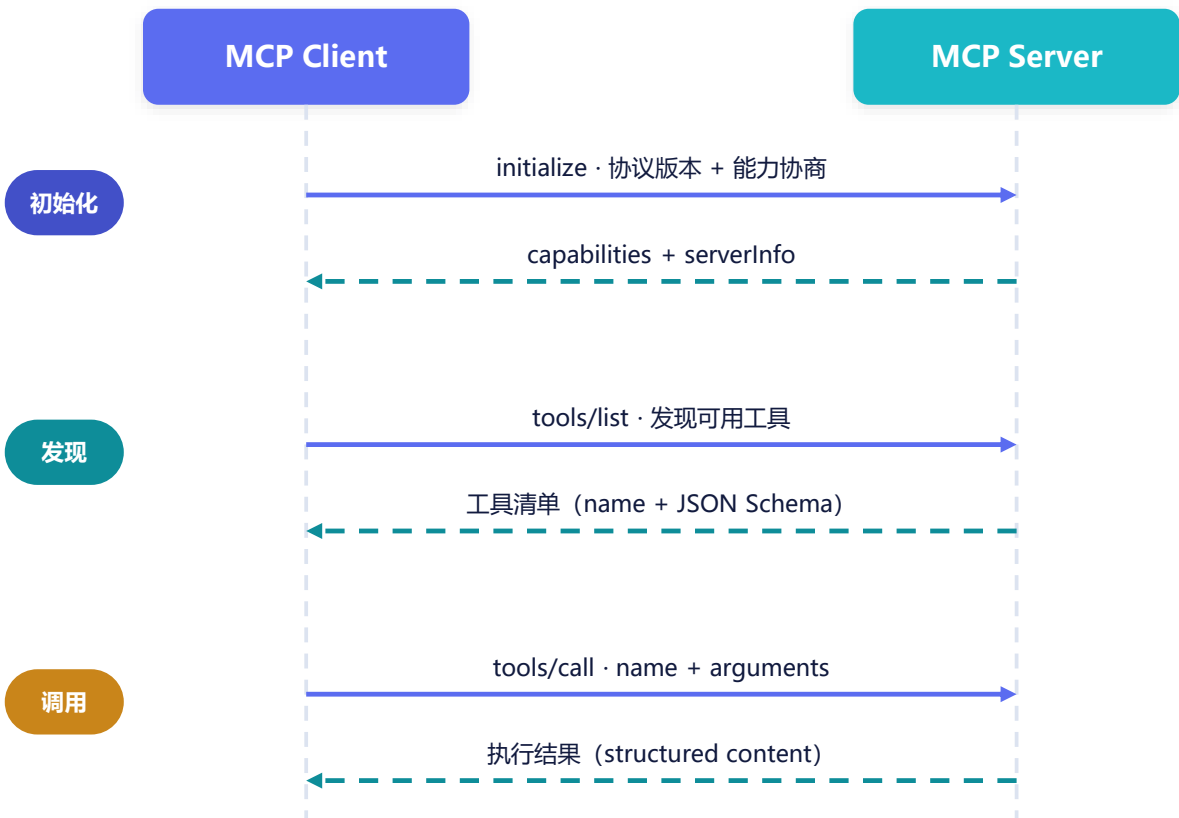
stdio 本地子进程, 单客户端、零网络配置

Streamable HTTP 远程、多客户端、可横向扩展, 配 OAuth 2.1 (2025-03 起取代旧版 SSE)



一次工具调用的完整时序 + 协议版本演进与治理

一次工具调用的时序 (JSON-RPC 2.0 over stdio / HTTP)



协议版本演进

- 2024-11 首发: Tools / Resources / Prompts 三原语
- 2025-03 Streamable HTTP 取代旧版 SSE
- 2025-06 OAuth 2.1 · 结构化输出 · elicitation
- 2025-11 注册中心 + 治理 (当前稳定版)
- 2026-07 RC: 无状态内核 · MCP Apps · Tasks

治理与规模

2025-12 捐赠给 Linux 基金会 / AAIF (Anthropic·Block·OpenAI 共同发起)
; 月均 9700 万+ SDK 下载、500+ 公开 Server。

用例 零售库存 Agent 用 Resources 读实时库存表、用 Tools 触发补货下单
——换数据源只换 Server, Agent 侧零改动。



Agent Card 发现 + Task 生命周期状态机

A2A 如何工作 (HTTP + JSON-RPC 2.0 + SSE)



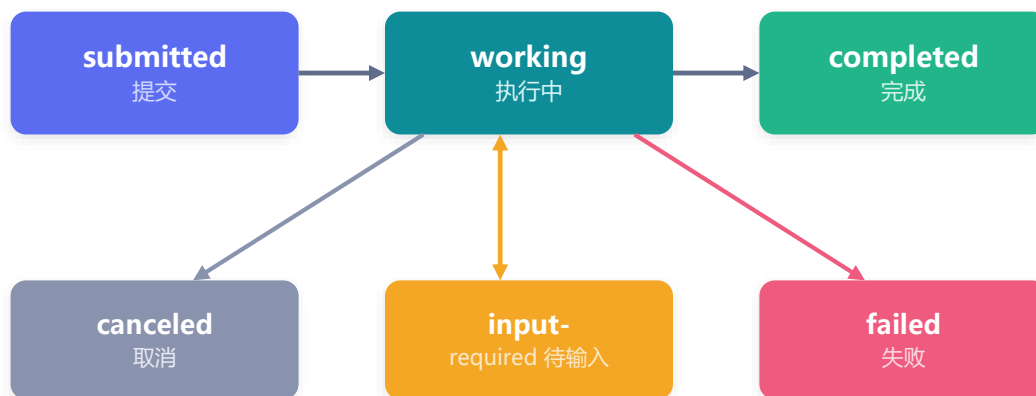
Agent Card

`/.well-known/agent-card.json`

- **能力 capabilities** 支持流式 / 推送 / 状态历史
- **技能 skills** 可处理的任务类型与示例
- **模态 modalities** 文本 / 文件 / 结构化数据
- **认证 auth** OAuth2 / API Key / mTLS
- **端点 endpoint** 服务 URL 与协议版本

客户端据此判断: 能不能干、怎么调、如何鉴权

Task 生命周期状态机



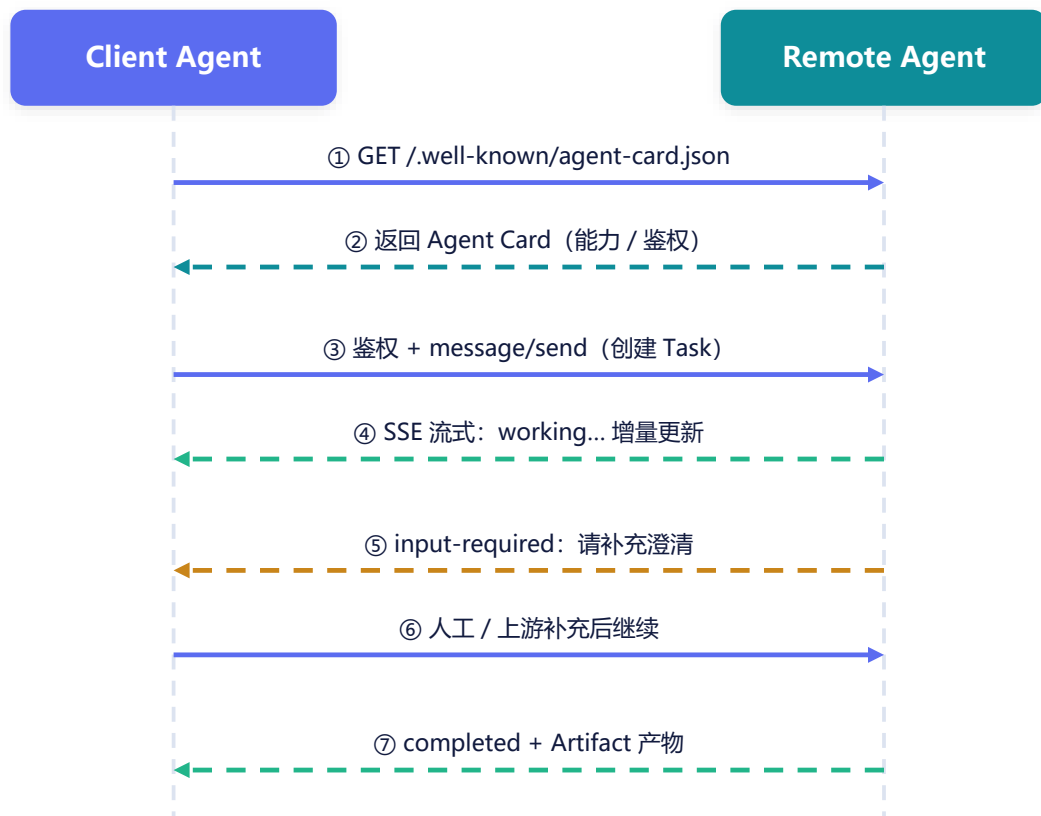
终止态 completed / canceled / failed; **input-required** 可暂停等待人工或上游补充后继续。

产物 任务结果以 Artifacts 形式返回——可为消息、文件或结构化数据，支持流式分块增量推送。



交互时序 · 三种更新获取方式 · 跨 Agent 协作用例

客户端 Agent 调用远端 Agent 的时序



三种获取更新的方式



轮询 Polling

tasks/get 周期查状态——最简单、实时性弱



SSE 流式 Streaming

长连接增量推送——交互式、低延迟首选



Webhook 推送

push notification 回调——长任务 / 离线场景

用例：跨 Agent 链式协作

库存 Agent

下单 Agent

供应商 Agent

用 A2A 横向串联三个自治 Agent；每个 Agent 内部各自用 MCP 连自己的数据库与系统——两套协议正交叠加。



MCP 与 A2A 不是竞争，是正交互补——生产基线是两者并用



类比：分层而非互斥

就像 Web 同时有 **HTTP / WebSocket / gRPC** —— 各司其职、长期共存。

MCP 解决「Agent 如何用工具 / 取数据」

A2A 解决「Agent 之间如何协作」

✓ 生产基线

真实系统里两者同时存在：Agent 向下用 MCP 接工具、向右用 A2A 接其他 Agent。

选型时不必二选一——先用 MCP 打通工具层，再用 A2A 做跨团队 / 跨组织的 Agent 协作。



MCP / A2A / ACP / ANP 对比 + 2025 收敛格局

协议 / 出品方	定位	传输	发现机制	身份与安全	成熟度
MCP Anthropic	Agent ↔ 工具/数据	JSON-RPC 2.0 stdio / HTTP	Server 能力声明 tools/list	OAuth 2.1 (远程)	事实标准 生态最大
A2A Google	Agent ↔ Agent	HTTP + JSON-RPC + SSE	Agent Card .well-known	OAuth2 / Key mTLS	快速成长 v1.0 已发
ACP IBM/BeeAI	Agent ↔ Agent (REST 原生)	REST / HTTP	Registry 经纪式	企业级 鉴权	已并入 A2A (2025-08)
ANP 社区	去中心化 开放网络	HTTP + JSON-LD	W3C DID 开放发现	DID / 可验证 凭证	早期 基础设施待熟



收敛而非混战 2025-08 ACP 并入 A2A；MCP 与 A2A 双双交由 Linux 基金会中立治理。格局趋于「MCP 接工具 + A2A 接 Agent」双协议基线，ANP 面向更远的去中心化场景。



开放协议带来的新攻击面与防御实践



主要风险

工具投毒 / 提示注入



恶意 Server 在工具描述或返回里植入指令，劫持模型行为

过度授权



Agent 拿到超出任务所需的权限，一次失误即放大为越权操作

跨组织身份伪造



A2A 跨信任域调用时，对方 Agent 身份难以验证，易被冒充

会话劫持 / 重放



远程传输中令牌泄露或被重放，攻击者接管有状态会话



防御实践



OAuth 2.1 + PKCE

远程 MCP 强制授权码 + PKCE，校验 RFC 9207 的 iss 防混淆



最小权限

按任务授予最小工具集与作用域，敏感动作单独审批



人在回路确认

高风险操作（下单 / 删除 / 转账）触发 human-in-the-loop



可验证身份

跨域用 DID / mTLS 双向认证，Agent Card 校验签名



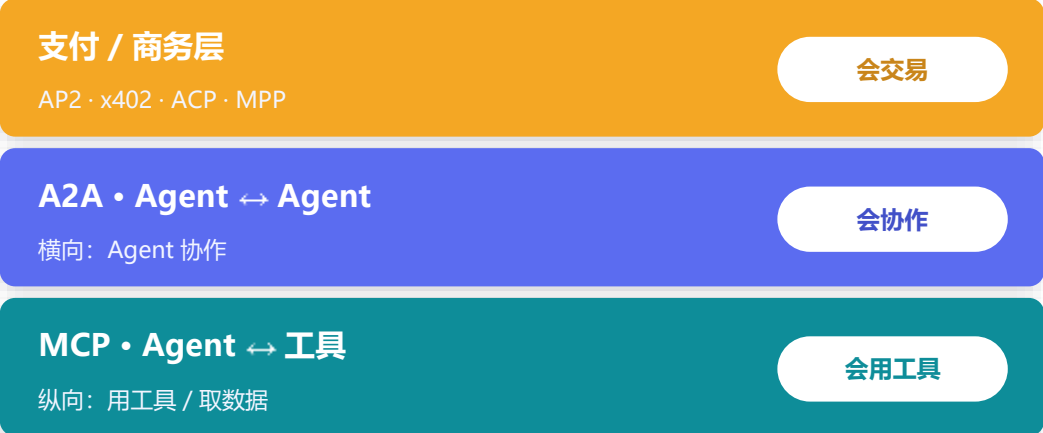
可观测与审计

全链路 trace (OpenTelemetry)，留存调用与决策证据



协议让 Agent 会对话、能用工具——再加一层「会交易」

协议栈：在 MCP / A2A 之上叠加经济层



解锁了什么

按调用付费 pay-per-call API	自动采购 代订 / 比价 / 下单
Agent ↔ 商家结账 agentic checkout	机器对机器商务 M2M / 机器经济
市场信号 麦肯锡：到 2030 年 agentic commerce 可影响全球 3–5 万亿美元 商务。	

支付 / 商务协议 (2025–2026 涌现)



注意：此 ACP = Agentic Commerce Protocol, 与前文 IBM 的 ACP (Agent Communication Protocol, 已并入 A2A) 同名不同物。



身份 KYA · 钱包护栏 · 风险与治理



身份与信任

Know Your Agent (KYA)

为 Agent 赋予可验证身份：DID / OAuth · OIDC 证明，跨组织调用时确认「这是谁、代表谁」。

Mandates 授权凭证

把「用户意图」密码学绑定到每笔交易：限定范围、限额、有效期与允许条件——下游授权来自显式同意，而非模型臆测。



钱包与护栏

会话上限 / 单笔限额

按额度与频率封顶

操作白名单

只允许预批的对象与动作

多方审批

大额需人工 / 多签确认

全程审计

每笔留痕、可回溯

云托管 AWS Bedrock AgentCore Payments：策略化限额 + 钱包管理 + 审计，开箱即用。



主要风险

超额消费 / 失控采购

权限过大、循环下单

欺诈与重放

支付凭证被盗用 / 重放

问责归属

出错时 mandate 谁负责

结算争议 + 跨域监管

退款 / 合规 / 司法差异

提示注入 → 越权付款

恶意内容诱导转账



愿景与冷静
接动钱。

「机器经济」雏形已现（x402 上线约 9 个月处理 1 亿+ 笔）；但落地铁律是：最小授权 + 人在回路 + 硬性限额 + 可审计 + 法务合规先行——别让 Agent 在没有硬边界时直

06



FRONTIER · 2026 前沿与工程实践

收敛、失败模式与上下文工程

Convergence, Failure Modes & Context Engineering





06 · 标准收敛

2024–2026：从协议爆发到中立治理与双协议基线



协议进入云原生与平台层

主流平台把 MCP / A2A 作为一等公民内建——Agent 互联从「自研对接」变为「平台默认能力」

Azure AI Foundry

微软

Bedrock AgentCore

AWS

Vertex Agent Builder

Google

Copilot Studio

M365



06 · 冷静的现实

为什么多智能体系统会失败？MAST 大规模实证

14 种失败模式 · 三大类占比

MAST · UC Berkeley · NeurIPS 2025 | 1642 条轨迹 · 7 个框架

系统设计缺陷

System Design



角色定义不清、流程编排错误、规范缺失

智能体间错位

Inter-Agent Misalignment



沟通错位、信息不同步、相互误解

任务验证不足

Task Verification



缺少正确性校验与终止判断，错误被放行

各框架失败率 41% - 86.7%：多智能体并非「越多越好」



核心洞察

多数失败不是模型不够强，而是**协作的工程问题**——角色、流程、验证没设计好。

能塞进单个上下文的任务，单个 LLM 往往优于多 Agent 协作。



真正的瓶颈：上下文

多 Agent 之所以诱人，常是为了「绕开单个上下文窗口的限制」。于是问题转化为——

如何做好上下文工程 (Context Engineering)

→ 下一页给出可落地的工程原则



上下文工程 · 何时该用多智能体 · 研究方向



上下文工程

隔离 Isolation

每个子 Agent 独立上下文窗口，互不污染（Anthropic 经验）

压缩 Compression

长任务定期摘要 / 记忆落盘，控制 token 与漂移

共享记忆 Memory

计划与中间结论写入外部存储，跨 Agent 可检索

显式委派 Delegation

下发明确目标、边界、输出格式与可用工具



何时才该上多智能体

适合

任务可并行、可分解为独立子问题

只读 / 研究型，子结果易聚合

上下文远超单窗口、需横向扩展

跨组织 / 跨团队的能力组合

谨慎

强耦合、需频繁同步状态的任务

能一次性放进单上下文的任务

对延迟 / 成本敏感（多 Agent ~15× token）

正确性要求高但缺乏验证手段



研究前沿

agentic RL

以最终任务结果为奖励，端到端训练 Agent 的规划与工具使用

评测体系

GAIA · AgentBench · HAL；LLM-as-judge 的可靠性

身份与治理

Agent 的 DID 身份、可验证凭证、跨域信任

可观测性

OpenTelemetry · W3C Trace Context 全链路追踪



06 · 工业界经验（一）

Anthropic 与 OpenAI 的方法论：先简单，后复杂



Anthropic

《Building Effective Agents》

● 区分 workflow vs Agent

workflow=预定义代码编排；Agent=模型自主决定流程与工具调用

● 黄金法则

找到最简单的方案，只在必要时才增加复杂度

● 五种可组合模式

Prompt Chaining · Routing · Parallelization · Orchestrator-Workers · Evaluator-Optimizer

● 先直接用 API

框架的抽象会隐藏 prompt、增加调试难度；先吃透底层



OpenAI

《A Practical Guide to Building Agents》

● 先最大化单 Agent

把「单 Agent + 工具」能力做到位，非必要不拆分

● 多体建模为图

Manager (agent 即 tool, 边=工具调用) / Decentralized (handoff, 边=控制转移)

● 何时拆分

复杂条件逻辑、工具过载——>15 个清晰工具可行，<10 个重叠就乱

● 守门 guardrails

相关性 / 安全 / PII / 审核 / 工具风险分级 + 人在回路



共识 两家都强调——先用最简单的方案、Agent 只在规则失效时才上、高质量 prompt 与上下文是第一性。



单体还是多体? Cognition 与 Anthropic 的两种实践



Cognition (Devin)

反对盲目多体

《Don't Build Multi-Agents》

● 原则一 · 共享上下文

把完整对话 / 动作轨迹传给每个 Agent, 缺一不可

● 原则二 · 动作隐含决策

并行 Agent 各自假设 → 冲突决策 → 系统失败

● 推荐架构

单线程线性 Agent + 上下文压缩 (专门的摘要模型)



Anthropic

读密集用多体

多智能体研究系统

● Orchestrator-Worker

主管规划, 并行派发 3-5 个子 Agent, 各自独立上下文

● 显式委派 + 独立核对

下发目标 / 边界 / 输出格式 / 工具, 单独引用核对

● 效果与代价

内部评测 +90.2% vs 单体 Opus 4, 但 ~15× token



调和: 真正的分水岭不是「单 / 多」, 而是「读 vs 写」

读 / 研究类

可并行、易聚合 → 多体更优

写 / 编码 / 编辑

强耦合、需一致 → 单体更稳

混合任务

把「读阶段」「写阶段」架构分开



06 · 评测

怎么衡量 Agent? 主流基准、关键指标与评测陷阱

SWE-bench Verified

真实 GitHub issue 修复 (约 2%→80%+, 23→26)

GAIA

通用助手 · 多步工具 + 推理 · 三难度

τ -bench / τ^2 -bench

工具-Agent-用户 · 策略遵从 · pass^k

WebArena / OSWorld

真实网页 / 桌面操作 (人类≈78%)

AgentBench

多领域综合鲁棒性

METR Time Horizons

能 50% 完成的最长任务时长

指标: pass^1 看平均, pass^k 看可靠性



τ -bench: GPT-4o $\text{pass}^1 \sim 60\% \rightarrow \text{pass}^8 < 25\%$ (同一问题连续 8 次全部成功的概率)

评测陷阱

脚手架放大 同一 GAIA 任务因 scaffold 不同可差 30-50 分

基准不严谨 τ -bench 把空回答计为成功, trivial agent 拿 38%

可被刷分 2026 UC Berkeley 把 8 大基准 reward-hack 到 ~100%

LLM-as-judge 偏差 位置 / 长度 / 迎合偏好; 难题错误率 > 50%

污染 + 单次报告 分数虚高 5-15 分

结论 榜单是方向信号、不是 SLA。重点看 pass^k 可靠性 + 自建领域评测 + 成本 / 延迟, 而非单一漂亮分。

基准各测一面, 没有单一指标能代表「好 Agent」



多智能体与协议落地的高频陷阱 → 对策



过早上多智能体



→ 先工作流 / 单 Agent, 规则失效再加 agency



并行「写」操作互相冲突



→ 写任务单线程化, 多体只做读 / 智能不做写



工具又多又重叠



→ 工具少而清晰、边界不重叠, 按风险分级



迷信公开榜单



→ 可被刷分 / 脚手架放大, 当方向信号即可



子 Agent 互相不知道对方在做什么



→ 传完整轨迹 / 共享记忆, 显式同步状态



依赖框架黑盒、看不到 prompt



→ 直接用 API 起步, 吃透底层再上框架



只看 pass¹ 漂亮分



→ 看 pass^k 可靠性 + 自建领域评测



协议默认全开、无人在回路



→ 最小权限 + 高风险动作人工确认 + 全链路审计



KEY TAKEAWAYS

五个带走的结论

1

多智能体是手段，不是目的

先把单 Agent + 上下文工程做扎实；MAST 实证显示多数失败是协作的工程问题，不是模型不行。

2

三层协议栈：用工具 · 接 Agent · 会交易

纵向 MCP 连工具、横向 A2A 连 Agent，再叠加 AP2 / x402 支付层——Agent 从「会协作」走向「会交易」。

3

标准正在收敛，贴主流最安全

ACP 已并入 A2A，MCP 与 A2A 双双交由 Linux 基金会治理，并被主流云平台内建为默认能力。

4

编排与组织决定上限

拓扑（主管-工作者 / 层级 / 群体）与组织（角色 / 规范 / SOP）各有取舍；协调有真实开销，团队 3-4 个 Agent 最实用。

5

开放协议要显式设计安全

OAuth 2.1 + 最小权限 + 人在回路 + 可验证身份 + 全链路审计，默认全开就是事故。

Q & A

Thank You

延伸阅读

- 协议综述 [arXiv:2505.02279](#) (MCP / ACP / A2A / ANP)
- 失败模式 [MAST](#), [arXiv:2503.13657](#) (NeurIPS 2025)
- 工程实践 Anthropic: [构建多智能体研究系统](#)
- 规范文档 [modelcontextprotocol.io](#) · [a2a-protocol.org](#)

- ## 延伸阅读
- 协议综述 [arXiv:2505.02279](#) (MCP / ACP / A2A / ANP)
 - 失败模式 [MAST](#), [arXiv:2503.13657](#) (NeurIPS 2025)
 - 工程实践 Anthropic: [构建多智能体研究系统](#)
 - 规范文档 [modelcontextprotocol.io](#) · [a2a-protocol.org](#)

